# API : La structure d'une requête HTTP

**1. Requête envoyée par le client**

| | |
|---|---|
| **Première ligne** | POST /api/users HTTP/1.1 |
| **En-têtes (Headers)** | Host: example.com<br>Content-Type: application/json<br>Authorization: Bearer your_access_token<br>Content-Length: 56 |
| **Corps de la requête (Body)** : | { "nom": "Dubois Pierre",<br>"email": "pierre.dubois@saint-remi.net",<br>"password": "R5'zfrtr!rgtToo'5^r6"<br>} |

**La première ligne** est de la fome <methode> <uri> <protocol>
La méthode POST, qui signifie que l'on envoie des données au serveur.
Le chemin vers la ressource à laquelle nous voulons accéder **/api/users**
La version du protocole HTTP utilisée : **HTTP/1.1**

**Les En-têtes** comportent des couples clé : valeur
**Host** : Spécifie le domaine du serveur avec lequel la requête est en cours de communication. Ici, `example.com`.
**Content-Type** indique le type de données envoyées dans le corps de la requête. Dans ce cas, `application/json` signifie que les données sont au format JSON.
**Authorization** contient les informations nécessaires pour l'authentification. Ici, un **token Bearer**
**Content-Length** : Indique la longueur en octets du corps de la requête. Dans cet exemple, le corps JSON contient 56 caractères, donc la longueur est 56.

**2. Réponse envoyée par le serveur**

| | |
|---|---|
| **Première ligne** | HTTP/1.1 **201** Created |
| **En-têtes (Headers)** | Content-Type: application/json<br>Location: /api/users/1234<br>Content-Length: 85 |
| **Corps de la requête (Body)** : | { "message": "Utilisateur créé avec succès",<br>"user": {<br>"id": 1234,<br>"nom": "Dubois Pierre",<br>"email": "pierre.dubois@saint-remi.net"<br>}<br>} |

La première ligne comporte le code statut de la réponse : 201
L'entête indique le format de la réponse, l'url pour consulter la personne créée
Le corps de la réponse contient un message de succès et les détails de l'utilisateur, incluant l'ID généré et les informations envoyées dans la requête.
Si le serveur rencontre un problème avec les données envoyées :

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
Content-Length: 92
{ "error": "Données invalides",
  "message": "Le mot de passe doit contenir au moins 8 caractères et un caractère spécial."
}
```

## 3. Les principales méthodes

### 3.1. GET

La méthode GET permet de récupérer une ressource du serveur sans la modifier.
C'est la méthode la plus courante pour récupérer des données (par exemple, pour afficher une page web).
Exemple : GET /index.html

### 3.2. POST

La méthode POST est utilisée pour envoyer des données au serveur, souvent pour créer ou mettre à jour une ressource.
Elle est utilisée lors de l'envoi de formulaires ou de données dans des requêtes API.

### 3.3. PUT

La méthode PUT permet de mettre à jour une ressource existante ou de créer une nouvelle ressource si elle n'existe pas.
Elle est généralement utilisée pour remplacer ou mettre à jour une ressource sur le serveur.

### 3.4. 4. DELETE

La méthode DELETE est utilisée pour supprimer une ressource sur le serveur.
Elle est employée pour supprimer des éléments sur le serveur, comme la suppression d'un enregistrement dans une table.
Exemple : DELETE /user/123 pour supprimer l'utilisateur avec l'ID 123.

### 3.5. 5. HEAD

Description : La méthode HEAD est similaire à GET, mais elle ne récupère que les en-têtes de la réponse, sans le corps de la ressource.
Elle est utilisée pour vérifier les informations sur la ressource, comme ses métadonnées (par exemple, la date de modification).

### 3.6. 6. PATCH

La méthode PATCH est utilisée pour effectuer une mise à jour partielle d'une ressource.
Elle est utile lorsque vous ne voulez pas envoyer l'intégralité d'une ressource, mais seulement une partie de celle-ci pour la mettre à jour.
Exemple PATCH /user/123 pour mettre à jour une partie des informations de l'utilisateur avec l'ID 123.

### 3.7. 7. OPTIONS

La méthode OPTIONS permet de demander au serveur quelles sont les méthodes HTTP autorisées pour une ressource spécifique. Elle est utilisée pour vérifier les capacités du serveur ou les options disponibles pour une ressource donnée.
La méthode OPTIONS est particulièrement utilisée dans le contexte des requêtes Cross-Origin Resource Sharing (CORS). Elle permet de découvrir les méthodes et en-têtes que le serveur accepte avant de procéder à une requête réelle (souvent utilisée dans les API REST pour les prérequis CORS).
Dans le contexte de CORS, un navigateur enverra une requête OPTIONS avant une requête HTTP complexe (comme POST ou PUT) pour vérifier que le serveur autorise la requête à partir de l'origine du client. Cela permet d'éviter que des requêtes potentiellement dangereuses soient envoyées sans autorisation explicite du serveur.

**4. Envoi d'une requête à l'aide d'un langage de programmation**

En Python

```
import requests

# Envoi d'une requête GET
response = requests.get('https://example.com')

# Vérification du code de statut HTTP
print(response.status_code)

# Affichage du contenu de la réponse
print(response.text)
```

En Javascript

```
fetch('https://example.com')
  .then(response => response.text())
  .then(data => {
    console.log(data);  // Affiche le contenu de la réponse (par exemple HTML)
  })
  .catch(error => {
    console.error('Erreur:', error);
  });
```

En PHP

```
<?php
$url = "https://example.com";
$ch = curl_init($url);

// Définir les options de la requête
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);

// Exécuter la requête
$response = curl_exec($ch);

// Vérifier si une erreur s'est produite
if(curl_errno($ch)) {
    echo 'Erreur : ' . curl_error($ch);
} else {
    // Afficher la réponse
    echo $response;
}

// Fermer la session cURL
curl_close($ch);
```